

Generative Models: VAEs, GANs, and Diffusion

KAUST–Mawhiba IOAI Training

Week 4 · Day 24



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 17, 2026

Part 1: VAEs

1. Bridge & Motivation
2. From AE to VAE
3. VAE Loss & Training
4. VAE Results

Part 2: GANs

5. The Adversarial Game
6. GAN Training & Results
7. Conditional & StyleGAN

Part 3: Diffusion

8. The Noise Idea
9. Forward & Reverse Process
10. Training & Sampling
11. Stable Diffusion
12. Summary

Discriminative models (all previous days):

- ▶ **Day 9:** CNN classifies images
- ▶ **Day 10:** Transfer learning reuses features
- ▶ **Day 19:** ViT + LoRA fine-tunes efficiently
- ▶ **Day 23:** CLIP matches images to text

These models answer: *what is in this image?*

Today: Generative models

- ▶ Create *new* images from scratch
- ▶ Turn text into photorealistic pictures
- ▶ Edit, style-transfer, upscale images
- ▶ Imagine things that never existed

These models answer: *what would this look like?*

Today's question: How do you teach a computer to *create* images, not just classify them?

► **Classification** is relatively easy:

- Input: $96 \times 96 \times 3$ image $\rightarrow$ Output: 1 number (class label)
- Many-to-one mapping — lots of valid inputs for each class

► **Generation** is incredibly hard:

- Input: random noise or text $\rightarrow$ Output: $512 \times 512 \times 3$ image (786,432 numbers!)
- One-to-many mapping — and most combinations of pixels are garbage
- The space of “valid images” is a tiny island in the ocean of possible pixel values

► **The key insight:** We need to learn the *distribution* of real images

- What makes a face look like a face? A flower like a flower?
- If we learn this distribution, we can *sample* from it to generate new images

- ▶ **Day 11:** We built autoencoders — compress an image to a small code, then reconstruct it
 - Encoder: image (784 pixels) $\rightarrow$ latent code z (e.g., 32 numbers)
 - Decoder: latent code $z \rightarrow$ reconstructed image
 - Trained with MSE: make output look like input
- ▶ **The problem we identified on Day 11:**
 - AEs reconstruct well, but their latent space is *messy*
 - If you pick a random point in latent space $\rightarrow$ garbage output
 - Nearby points in latent space might produce very different images
 - **You cannot use a standard AE to generate new images**
- ▶ **Today's solution:** VAEs fix this by forcing the latent space to be *smooth and organized*

Part 1: VAEs

1. From AE to VAE
2. The KL divergence trick
3. Reparameterization trick
4. VAE results and limitations

Part 2: GANs

5. The adversarial game
6. GAN training
7. Conditional GANs
8. StyleGAN

Part 3: Diffusion Models

9. The noise-and-denoise idea
10. Forward and reverse processes
11. Training diffusion models
12. Stable Diffusion = CLIP + Diffusion
13. Comparison and summary

Running Example

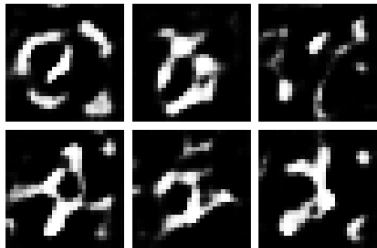
We will generate faces throughout today. Same task, three different approaches — watch how the quality and control improves from VAEs to GANs to Diffusion.

- ▶ Encoder compresses image to latent code z
- ▶ Decoder reconstructs image from z
- ▶ Loss = $\text{MSE}(\text{input}, \text{output})$
- ▶ Great for *compression* and *denoising*

But can we generate new images?

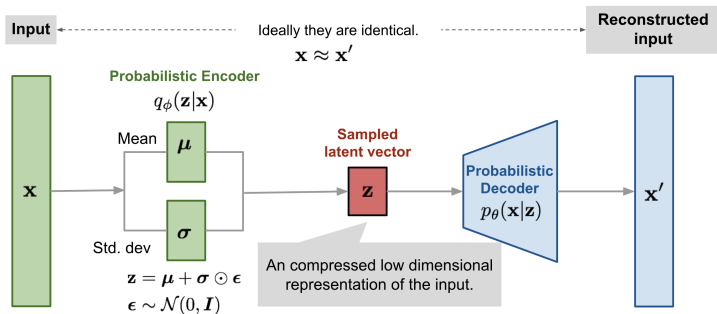
- ▶ Sample a random z and decode it?
- ▶ *No!* The latent space is messy and full of gaps
- ▶ Random points decode to garbage

generated_images



Sampling from an AE latent space:
mostly garbage

- ▶ **Standard AE:** input $\rightarrow$ encoder $\rightarrow$ single point $z \rightarrow$ decoder $\rightarrow$ output
- ▶ **VAE:** input $\rightarrow$ encoder $\rightarrow (\mu, \sigma) \rightarrow$ sample $z \sim \mathcal{N}(\mu, \sigma^2) \rightarrow$ decoder $\rightarrow$ output
- ▶ The encoder no longer predicts a single code — it predicts a *Gaussian distribution*
- ▶ We *force* this distribution to stay close to $\mathcal{N}(0, 1)$ via KL divergence
- ▶ **Result:** The latent space becomes smooth and complete
 - Nearby points decode to similar images
 - Every region of the space produces something reasonable
 - We can now generate! Just sample $z \sim \mathcal{N}(0, 1)$ and decode



- ▶ Encoder outputs μ and $\log \sigma^2$ (two vectors, not one)
- ▶ Sample $z = \mu + \sigma \cdot \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$
- ▶ Decoder reconstructs from the sampled z

Training time:

1. Encode image $\rightarrow$ get μ, σ
2. Sample $z \sim \mathcal{N}(\mu, \sigma^2)$
3. Decode $z \rightarrow$ reconstruct image
4. Loss forces $(\mu, \sigma) \approx (0, 1)$

Generation time:

1. Skip the encoder entirely!
2. Sample $z \sim \mathcal{N}(0, 1)$
3. Decode $z \rightarrow$ new image!

Works because the decoder learned to handle inputs from $\mathcal{N}(0, 1)$

Key Insight

The KL divergence term ensures the encoder maps everything near $\mathcal{N}(0, 1)$. So at generation time, **any** sample from $\mathcal{N}(0, 1)$ is in-distribution for the decoder.

- ▶ The VAE loss has two parts:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\|x - \hat{x}\|^2}_{\text{Reconstruction Loss}} + \underbrace{D_{\text{KL}}(q(z|x) \parallel \mathcal{N}(0, I))}_{\text{KL Divergence}}$$

- ▶ **Reconstruction loss:** Make the output look like the input (same as AE)
- ▶ **KL divergence:** Force the latent distribution to be close to $\mathcal{N}(0, 1)$
- ▶ These two terms *compete*:
 - Reconstruction alone $\rightarrow$ sharp images but messy latent space
 - KL alone $\rightarrow$ perfect Gaussian but decoder ignores z
 - Balance $\rightarrow$ good reconstruction AND smooth latent space

- ▶ D_{KL} measures how different two distributions are
- ▶ For the VAE: how far is $q(z|x) = \mathcal{N}(\mu, \sigma^2)$ from the prior $\mathcal{N}(0, 1)$?
- ▶ Closed-form solution (no sampling needed!):

$$D_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

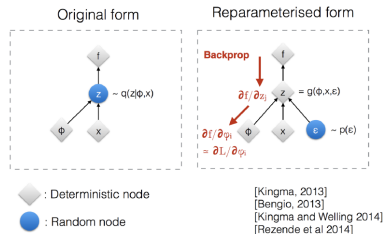
- ▶ This equals zero when $\mu = 0$ and $\sigma = 1$ (exactly the prior)
- ▶ Grows as μ moves away from 0 or σ deviates from 1
- ▶ **Intuition:** It is a penalty that says “don’t put your encodings too far from the center, and don’t make them too narrow or too wide”

Problem: Sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ is not differentiable — we cannot backpropagate through a random operation!

Solution: Reparameterize the sampling:

$$z = \mu + \sigma \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

- ▶ The randomness is now in ε (external to the network)
- ▶ μ and σ are deterministic outputs of the encoder
- ▶ Gradients flow through μ and σ normally



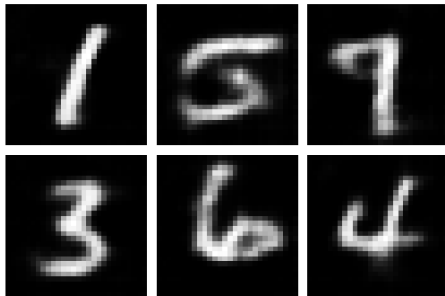
```
class VAE(nn.Module):
    def __init__(self, latent_dim=32):
        super().__init__()
        self.encoder = nn.Sequential(...)
        self.fc_mu = nn.Linear(hidden, latent_dim)
        self.fc_var = nn.Linear(hidden, latent_dim)
        self.decoder = nn.Sequential(...)

    def encode(self, x):
        h = self.encoder(x)
        return self.fc_mu(h), self.fc_var(h)

    def reparameterize(self, mu, log_var):
        std = torch.exp(0.5 * log_var)
        eps = torch.randn_like(std)      # sample noise
        return mu + std * eps            # the trick!

    def forward(self, x):
        mu, log_var = self.encode(x)
        z = self.reparameterize(mu, log_var)
        return self.decoder(z), mu, log_var
```

generated_images



- ▶ Sample $z \sim \mathcal{N}(0, 1)$, pass through decoder $\rightarrow$ new digits!
- ▶ Not perfect, but clearly recognizable — and all *novel* (not copies of training data)



- ▶ VAEs on faces: generates plausible but *blurry* faces
- ▶ This blurriness is a well-known limitation — MSE loss averages over possibilities



- ▶ **Linear interpolation:** encode two images to z_1, z_2 , walk between them:
 $z_t = (1-t) \cdot z_1 + t \cdot z_2$
- ▶ Every point decodes to a valid face — the latent space is *continuous*
- ▶ Standard AE would fail here (gaps $\rightarrow$ garbage between encodings)

Strengths:

- ▶ Principled probabilistic framework
- ▶ Smooth, interpretable latent space
- ▶ Can interpolate and do arithmetic
- ▶ Stable training (unlike GANs)
- ▶ Fast generation (single forward pass)

Limitations:

- ▶ Generated images are *blurry*
- ▶ MSE loss averages over modes
- ▶ Cannot produce sharp, photorealistic output
- ▶ Limited resolution

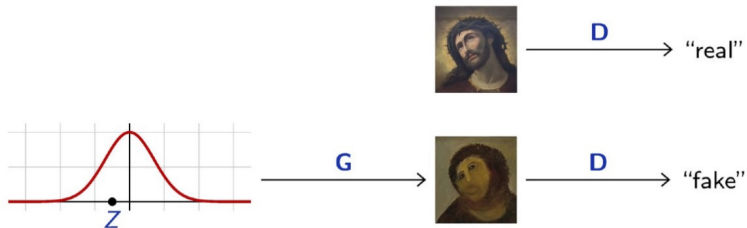
Can we do better?

- GANs produce sharp images!
- Diffusion models do both!

- ▶ Introduced by Ian Goodfellow et al. (2014)
- ▶ Train *two networks* that compete against each other:
 - **Generator G** : Takes random noise z and produces a fake image
 - **Discriminator D** : Looks at an image and decides: real or fake?
- ▶ The generator tries to fool the discriminator
- ▶ The discriminator tries not to be fooled
- ▶ Over time, G gets so good that D cannot tell the difference

Analogy

G = art forger making fake paintings. D = art expert detecting fakes. As the expert gets better, the forger must improve too — until the fakes are indistinguishable from originals.



- ▶ G maps noise $z \sim \mathcal{N}(0, 1)$ to an image
- ▶ D maps an image to a probability: $D(x) \in [0, 1]$
- ▶ $D(x) = 1$ means “definitely real”, $D(x) = 0$ means “definitely fake”

- ▶ The minimax game:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

- ▶ **Discriminator** maximizes: push $D(\text{real})$ toward 1, push $D(\text{fake})$ toward 0
- ▶ **Generator** minimizes: push $D(G(z))$ toward 1 (fool the discriminator)
- ▶ **In plain English:**
 - D wants to be a perfect lie detector
 - G wants to be a perfect liar
 - The equilibrium: G produces data so real that D outputs 0.5 for everything

VAE:

- ▶ Explicit probability model
- ▶ Optimizes reconstruction + KL
- ▶ Produces *blurry* but diverse images
- ▶ Stable training
- ▶ Has an encoder (can encode images)

GAN:

- ▶ Implicit model (no explicit density)
- ▶ Optimizes adversarial game
- ▶ Produces *sharp* but less diverse images
- ▶ Tricky training (can collapse)
- ▶ No encoder (only generates)

VAEs are blurry because MSE averages. GANs are sharp because the discriminator punishes anything that looks “off.”

1. **Train Discriminator** (a few steps):

- Sample real images from dataset
- Generate fake images with G (detach from G 's graph)
- Train D to output 1 for real, 0 for fake (binary cross-entropy)

2. **Train Generator** (one step):

- Generate fake images with G
- Pass through D — but now we want D to output 1!
- Backprop through D (frozen) into G

3. Repeat until convergence (or collapse...)

The key: D and G must stay balanced. If D gets too strong, G 's gradients vanish. If G gets too strong, D cannot learn.

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

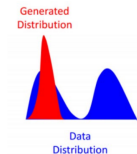
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Mode collapse:

- ▶ G finds one image that fools D and only produces that
- ▶ No diversity — all generated faces look the same



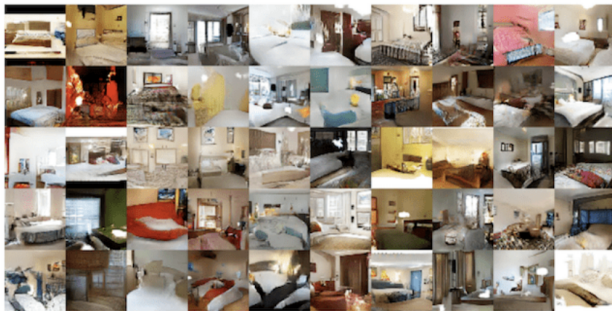
Training instability:

- ▶ Loss oscillates, does not converge
- ▶ Vanishing gradients when D is too good

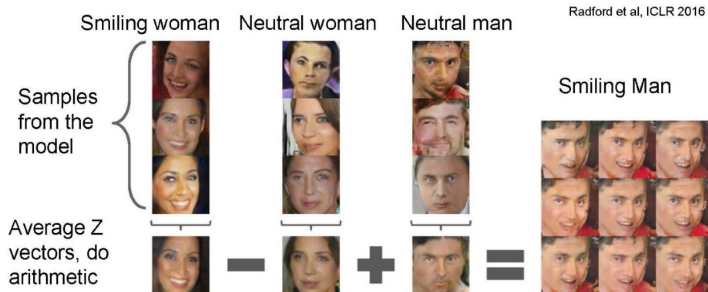
Solutions:

- ▶ WGAN: better loss function
- ▶ Spectral normalization
- ▶ Progressive growing / StyleGAN

GAN Results: The Evolution of Quality

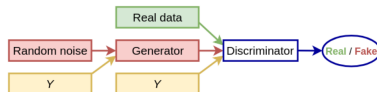


- ▶ 2014: blurry blobs. 2018: photorealistic faces at 1024×1024
- ▶ GANs went from toy examples to state-of-the-art in just 4 years
- ▶ The key: better architectures (DCGAN $\rightarrow$ ProGAN $\rightarrow$ StyleGAN)

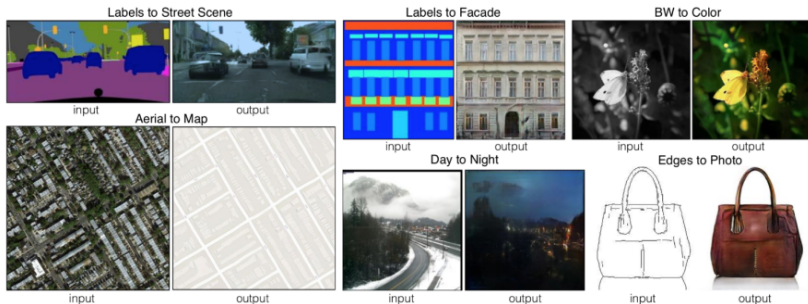


- ▶ Just like VAEs, GANs learn a smooth latent space
- ▶ Walk between two noise vectors $\rightarrow$ smooth transition between faces
- ▶ Every point along the path produces a valid, sharp image

- ▶ Standard GAN: noise $\rightarrow$ random image (no control)
- ▶ **Conditional GAN:** noise + *condition* $\rightarrow$ specific image
- ▶ The condition can be:
 - A class label ("generate a 7")
 - A text description
 - Another image (image-to-image)
- ▶ Both G and D receive the condition
- ▶ D checks: is this image real *and* does it match the condition?



Conditional GAN: Image-to-Image Translation



- ▶ **pix2pix** (Isola et al., 2017): condition = input image, output = transformed image
- ▶ Sketch $\rightarrow$ photo, day $\rightarrow$ night, edges $\rightarrow$ cats, labels $\rightarrow$ street scene
- ▶ Uses GAN loss + L1 reconstruction for sharp, faithful translations

- ▶ Karras et al. (NVIDIA, 2019)
- ▶ Key innovation: separate *style* at different scales
- ▶ **Coarse styles** (4–8px):
 - Face shape, pose, glasses
- ▶ **Medium styles** (16–32px):
 - Hair style, eyes open/closed
- ▶ **Fine styles** (64–1024px):
 - Hair color, skin texture



- ▶ Instead of feeding z directly, map through a learned network:

$$z \in \mathbb{R}^{512} \xrightarrow{\text{mapping net}} w \in \mathbb{R}^{512} \text{ (style code)}$$

- ▶ The W space is *disentangled*: one dimension = one attribute
 - Move along one direction $\rightarrow$ add glasses; another $\rightarrow$ change age
- ▶ Style injected at each layer via **AdaIN** (Adaptive Instance Norm):

$$\text{AdaIN}(x_i, y) = y_s \cdot \frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_b$$

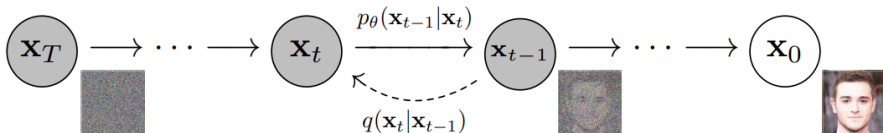
- ▶ Per-layer control: coarse layers get pose, fine layers get texture



These faces do not exist. Generated by StyleGAN at 1024×1024 .
Every one is unique, photorealistic, and controllable via the W space.

- ▶ GANs generate in *one shot*: noise $\rightarrow$ image (single forward pass)
- ▶ This is hard! The generator must learn the entire mapping at once
- ▶ **Diffusion models** take a different approach:
 - Generate in *many small steps* (typically 1000)
 - Each step removes a tiny bit of noise
 - Much easier to learn — each step is a small correction
- ▶ **Analogy:** Sculpting a statue
 - GAN: carve the whole statue in one move (hard!)
 - Diffusion: start with a block of marble, chip away bit by bit (easy steps!)

The Core Idea: Destroy, Then Learn to Rebuild



- ▶ **Forward process:** Gradually add noise to a clean image until it becomes pure static. This is easy — just add Gaussian noise at each step.
- ▶ **Reverse process:** Train a neural network to undo each noise step. Start from pure noise, denoise step by step $\rightarrow$ clean image emerges.
- ▶ The network only needs to learn one small denoising step — much simpler than generating from scratch!



- ▶ “Diffusion Models Beat GANs on Image Synthesis” (Dhariwal & Nichol, OpenAI, 2021)
- ▶ Better image quality AND better diversity (no mode collapse!)
- ▶ Stable training — no adversarial balancing act
- ▶ The downside: *slow* generation (1000 steps vs. 1 for GANs)

Quick Summary: Three Approaches

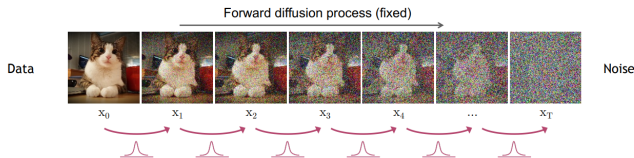
	VAE	GAN	Diffusion
Quality	Blurry	Sharp	Best
Diversity	High	Low (mode collapse)	High
Training	Stable	Unstable	Stable
Speed	Fast (1 pass)	Fast (1 pass)	Slow (1000 steps)
Year	2013	2014	2020

- ▶ Diffusion wins on quality and stability, but pays with speed
- ▶ In practice: Stable Diffusion generates a 512×512 image in ~ 5 seconds on a good GPU
- ▶ Ongoing research: faster samplers (DDIM, DPM-Solver) reduce to 20–50 steps

- Start with clean image x_0 . At each step t , add Gaussian noise:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

- β_t : noise schedule (0.0001 to 0.02). After $T \approx 1000$ steps: pure noise



- **Shortcut** — jump directly to step t (no need to compute all previous):

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$$

- ▶ **Goal:** Learn to reverse each step — go from x_t back to x_{t-1}
- ▶ A neural network $\varepsilon_\theta(x_t, t)$ predicts the noise that was added
- ▶ Then we can compute the “denoised” version:

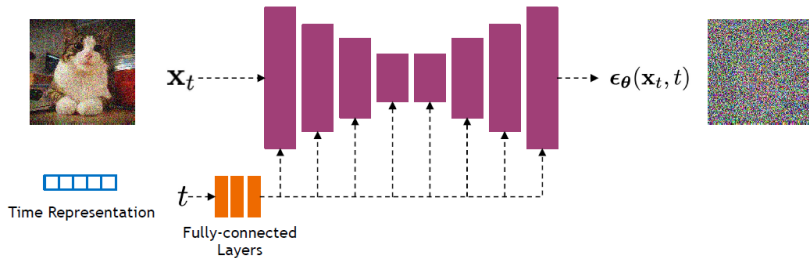
$$x_{t-1} = \frac{1}{\sqrt{1 - \beta_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t \cdot z$$

- ▶ The network takes two inputs: the noisy image x_t and the timestep t
- ▶ It outputs: the predicted noise $\hat{\varepsilon}$

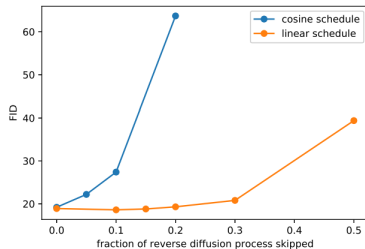
Intuition

The network answers: “Given this noisy image at step t , what noise was added?” Subtract the predicted noise $\rightarrow$ slightly cleaner image. Repeat 1000 times $\rightarrow$ clean image.

The Denoising Network: U-Net



- Architecture: **U-Net** with skip connections (same as segmentation from Day 11!)
- ResNet blocks + self-attention layers
- Timestep t injected via sinusoidal position embeddings (like Transformers!)
- Same network used for ALL timesteps (shared weights)



- ▶ **Linear schedule:** β_t increases linearly from 0.0001 to 0.02
 - Simple, but destroys image information too quickly at the end
- ▶ **Cosine schedule:** Smoother degradation, keeps more information longer
 - Better results in practice — used by most modern models

Algorithm:

1. Pick a clean image $\mathbf{x}_0$ from dataset
2. Pick a random timestep $t \in \{1, \dots, T\}$
3. Sample noise $\varepsilon \sim \mathcal{N}(0, I)$
4. Create noisy image:
$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$
5. Network predicts noise:
$$\hat{\varepsilon} = \varepsilon_{\theta}(\mathbf{x}_t, t)$$
6. Loss = MSE: $\|\varepsilon - \hat{\varepsilon}\|^2$

Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
$$\nabla_{\theta} \|\varepsilon - \varepsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, t)\|^2$$
 - 6: **until** converged
-

That's it! Just predict the noise. No adversarial training, no KL divergence. Plain MSE.

Algorithm:

1. Start from pure noise
 $x_T \sim \mathcal{N}(0, I)$
2. For $t = T, T-1, \dots, 1$:
 - 2.1 Predict noise: $\hat{\epsilon} = \epsilon_\theta(x_t, t)$
 - 2.2 Remove predicted noise (partially)
 - 2.3 Add a tiny bit of fresh noise (stochasticity)
3. Return x_0 (the final denoised image)

- ▶ Each step refines the image slightly — like a sculptor chipping away marble
- ▶ The same U-Net is called T times with different t values

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

```
# Training loop (simplified)
for x_0 in dataloader:
    # Random timestep for each image
    t = torch.randint(0, T, (batch_size,))

    # Sample noise
    noise = torch.randn_like(x_0)

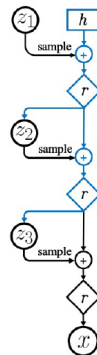
    # Create noisy image (one-step shortcut)
    x_t = sqrt_alpha_bar[t] * x_0 + sqrt_one_minus[t] * noise

    # Predict the noise
    predicted_noise = model(x_t, t)

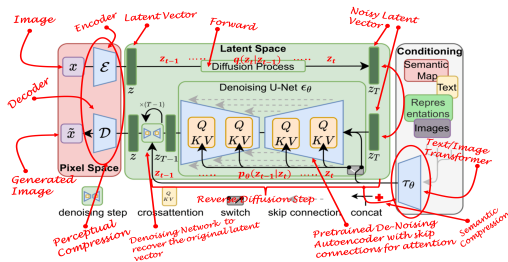
    # Simple MSE loss
    loss = F.mse_loss(predicted_noise, noise)
    loss.backward()
    optimizer.step()
```

Compare this to GAN training (two networks, adversarial balance, mode collapse)...
Diffusion training is almost boring in its simplicity!

- ▶ Diffusion models are actually a special case of *hierarchical VAEs*:
 - Each timestep is one “layer” of a very deep VAE
 - The encoder (forward process) is *fixed* — not learned
 - Latent variables have same dimension as the data
 - The decoder (reverse) is shared across all layers
- ▶ This connection explains why diffusion models are stable:
 - Like VAEs, they optimize a variational bound
 - Like VAEs, no adversarial training needed
 - Unlike VAEs, they avoid the blurriness (no MSE on pixels)



- ▶ Everything so far generates images from random noise — no control over *what* to generate
- ▶ **Text-to-image**: type a description, get a matching image
- ▶ This requires connecting *language understanding* with *image generation*
- ▶ The solution: combine two things you already know!
 - **CLIP** (Day 23): understands the relationship between text and images
 - **Diffusion models** (today): generates high-quality images
- ▶ **Stable Diffusion** = CLIP text encoder + diffusion in latent space



► Three components:

1. **Text encoder** (CLIP): converts text prompt to embedding
2. **U-Net** (diffusion): denoises in latent space, conditioned on text
3. **VAE decoder**: converts latent back to pixel space

► Key insight: run diffusion in a *compressed* latent space (64×64 instead of 512×512) — $64 \times$ less compute!

- ▶ The CLIP text encoder converts your prompt to an embedding vector
- ▶ This embedding is fed to the U-Net via *cross-attention*:
 - Queries: from the noisy image features
 - Keys/Values: from the text embedding
 - The image “attends” to relevant parts of the text at each denoising step
- ▶ **Classifier-free guidance:** Run the model twice per step
 - Once with the text condition (“a cat on a beach”)
 - Once without (unconditional)
 - Final prediction = unconditional + $w \cdot$ (conditional – unconditional)
 - $w > 1$ makes the model follow the text more strongly
- ▶ Higher w = more faithful to text but less diverse

DALL-E 2: Text-to-Image Results

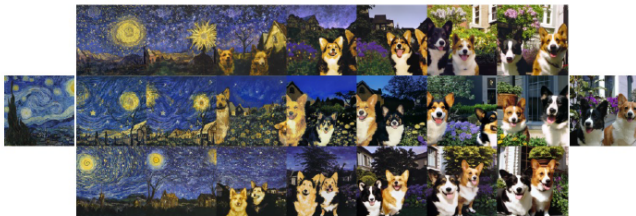


a shiba inu wearing a beret and black turtleneck



a close up of a handpalm with leaves growing from it

All generated from text descriptions. No real images involved.
DALL-E 2 uses CLIP embeddings + a diffusion model (OpenAI, 2022).



Interpolate image CLIP embeddings z_i

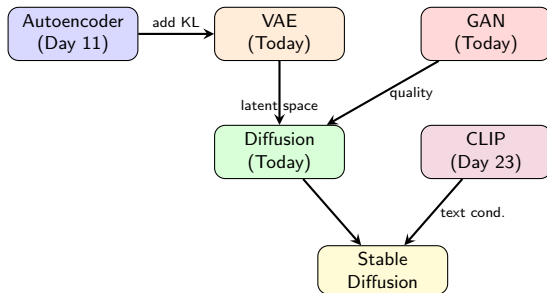
Use different x_T to get different interpolation trajectories.

- ▶ Interpolate between CLIP embeddings of two images
- ▶ The diffusion model generates smooth transitions between concepts
- ▶ Combines the interpolation power of VAEs with the quality of diffusion



“a man wearing a white hat”

- ▶ **Inpainting:** mask part of an image, describe what should fill it
- ▶ The diffusion model generates content that matches both the surrounding image AND the text description
- ▶ Remember inpainting from Day 23 SSL pretext tasks? Same idea, but now with text guidance!



- ▶ AE → VAE: add probabilistic latent space
- ▶ VAE → Stable Diffusion: compress to latent, then diffuse there
- ▶ CLIP → Stable Diffusion: provides text understanding
- ▶ GAN insights → Diffusion: adversarial training for the VAE decoder

VAEs

- ▶ AE + probabilistic latent
- ▶ Loss: recon + KL div
- ▶ Smooth latent space
- ▶ Can interpolate
- ▶ Blurry outputs

GANs

- ▶ Generator vs. Discriminator
- ▶ Adversarial minimax game
- ▶ Sharp outputs
- ▶ Mode collapse risk
- ▶ StyleGAN: fine control

Diffusion

- ▶ Add noise, learn to denoise
- ▶ Simple MSE training
- ▶ Best quality + diversity
- ▶ Slow (many steps)
- ▶ + CLIP = Stable Diffusion

1. What is the main difference between an AE and a VAE?
A. VAE uses a bigger network B. VAE encodes to a *distribution*, not a point
C. VAE uses a different loss D. VAE has no decoder
2. Why are VAE outputs blurry?
A. The network is too small B. MSE loss *averages* over possible outputs
C. The KL term is too strong D. Not enough training data
3. In a GAN, what does the generator try to do?
A. Classify images B. Reconstruct inputs
C. *Fool the discriminator* D. Compress images
4. What does the diffusion model's U-Net predict?
A. The clean image B. The *noise* that was added
C. The next timestep D. A class label

5. Why is Stable Diffusion faster than running diffusion on full-resolution images?
 - A. It uses fewer timesteps
 - B. It runs diffusion in a *compressed latent space* (64×64 instead of 512×512)
 - C. It uses a smaller U-Net
 - D. It skips some steps
6. What role does CLIP play in Stable Diffusion?
 - A. It generates the image
 - B. It *encodes the text prompt* so the model knows what to generate
 - C. It discriminates real from fake
 - D. It compresses the image
7. What is mode collapse in GANs?
 - A. The model crashes
 - B. The generator *only produces one type* of output (no diversity)
 - C. Training is too slow
 - D. The discriminator always wins

► **VAE loss:**

$$\mathcal{L} = \|x - \hat{x}\|^2 + D_{\text{KL}}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, I))$$

► **KL divergence (closed form):**

$$D_{\text{KL}} = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

► **GAN objective:**

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

► **Diffusion training loss:**

$$\mathcal{L} = \|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2$$

► **Reparameterization:** $z = \mu + \sigma \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$

- ▶ **Day 25 Lab:** You will generate images with a pre-trained diffusion model and explore CLIP-guided generation hands-on
- ▶ **The bigger picture:** Everything connects
 - Day 11 AEs → Today's VAEs → Stable Diffusion's latent space
 - Day 23 CLIP → Today's text conditioning
 - Day 23 contrastive learning → CLIP training → guides generation
 - Day 19 LoRA → fine-tune diffusion models (DreamBooth, LoRA for SD)
- ▶ **For the olympiad:** Understanding the VAE loss, the GAN objective, and how diffusion training works (predict the noise, MSE loss) are all fair game

Credits

KAUST Academy

King Abdullah University of Science and Technology